



WORKSHOP JAKARTA EE

Créer une API REST pour la gestion d'école

1. Objectifs du workshop

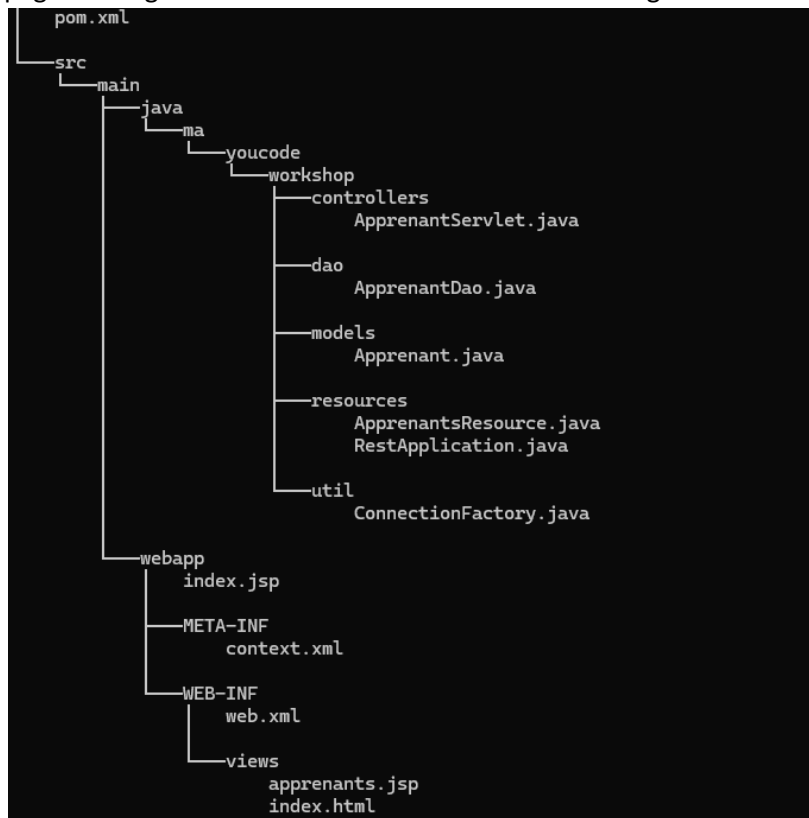
À la fin de ce workshop, vous serez capable de :

- expliquer la différence entre une **Servlet qui renvoie du HTML** (via une JSP) et une **API REST qui renvoie du JSON** ;
- ajouter **Jersey** (l'implémentation de référence de JAX-RS) à un projet Maven déployé sur Tomcat ;
- déclarer une application REST avec `@ApplicationPath` et une ressource avec `@Path`, `@GET`, `@Produces` ;
- **réutiliser le DAO existant** et le pool de connexions (DataSource JNDI) sans les modifier ;
- tester l'API avec le navigateur, `curl` ou Postman, et diagnostiquer les erreurs les plus courantes.

Astuce : l'idée clé du workshop : la couche données (Model + DAO + DataSource) ne change pas. On ajoute seulement une nouvelle « porte d'entrée » (la ressource REST) à côté de la Servlet existante.

2. Le projet de départ

Le projet `workshop-connection-db` vous est fourni. Il affiche déjà la liste des apprenants dans une page HTML grâce à une Servlet et une JSP. Voici son organisation :



3. Étape 0 : vérifier que le projet de départ fonctionne

Avant d'ajouter quoi que ce soit, assurez-vous que la version HTML marche. Sinon, vous ne saurez pas si une erreur vient de votre code REST ou de l'environnement.

1. Démarrez MySQL et vérifiez que la base `gestion_ecole` contient la table `etudiant` avec quelques lignes.
2. Démarrez **un seul** Tomcat 10.1.
3. Construisez le WAR : `mvn clean package`.
4. Copiez `target/workshop-connection-db.war` dans le dossier `webapps` de Tomcat.
5. Ouvrez `http://localhost:8080/workshop-connection-db/apprenants` : le tableau des apprenants s'affiche.

```
CREATE DATABASE IF NOT EXISTS gestion_ecole;
• USE gestion_ecole;
• CREATE TABLE IF NOT EXISTS etudiant (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    nom     VARCHAR(100) NOT NULL,
    prenom  VARCHAR(100) NOT NULL,
    email   VARCHAR(150),
    filiere VARCHAR(100)
);
• INSERT INTO etudiant (nom, prenom, email, filiere) VALUES
    ('Alaoui', 'Sara', 'sara@ecole.ma', 'Informatique'),
    ('Bennani', 'Youssef', 'youssef@ecole.ma', 'Gestion');
```

4. Étape 1 : comprendre REST et JAX-RS

Une **API REST** expose des **ressources** (ici : les apprenants) via des URL, et utilise les verbes HTTP pour dire ce qu'on veut en faire. Elle renvoie des données brutes (JSON), pas une page HTML : c'est le client (application mobile, front React/Angular, Postman...) qui décide comment les afficher.

Verbe HTTP	URL	Action	Dans ce workshop
GET	<code>/api/apprenants</code>	Lire tous les apprenants	À réaliser
GET	<code>/api/apprenants/{id}</code>	Lire un apprenant	À réaliser
POST	<code>/api/apprenants</code>	Créer un apprenant	À réaliser
PUT	<code>/api/apprenants/{id}</code>	Modifier	Bonus
DELETE	<code>/api/apprenants/{id}</code>	Supprimer	Bonus

JAX-RS (package `jakarta.ws.rs`) est la spécification Jakarta EE pour écrire des API REST avec des annotations. Tomcat n'est pas un serveur Jakarta EE complet : il ne fournit pas JAX-RS. On ajoute donc **Jersey**, son implémentation de référence, dans notre WAR.

Annotation	Rôle
<code>@ApplicationPath("/api")</code>	Préfixe commun à toutes les ressources REST de l'application
<code>@Path("/apprenants")</code>	URL de la ressource (s'ajoute après <code>/api</code>)
<code>@GET</code>	La méthode répond aux requêtes HTTP GET
<code>@Produces(MediaType.APPLICATION_JSON)</code>	Le format de la réponse est JSON

5. Étape 2 : ajouter Jersey dans le pom.xml

Ajoutez ces trois dépendances dans la section `<dependencies>` du `pom.xml` :

```
23     <dependency>
24         <groupId>org.glassfish.jersey.containers</groupId>
25         <artifactId>jersey-container-servlet</artifactId>
26         <version>3.1.9</version>
27     </dependency>
28
29     <dependency>
30         <groupId>org.glassfish.jersey.inject</groupId>
31         <artifactId>jersey-hk2</artifactId>
32         <version>3.1.9</version>
33     </dependency>
34
35     <dependency>
36         <groupId>org.glassfish.jersey.media</groupId>
37         <artifactId>jersey-media-json-binding</artifactId>
38         <version>3.1.9</version>
39     </dependency>
40
```

Dépendance	Sans elle...
jersey-container-servlet	les annotations <code>@Path</code> , <code>@GET</code> ne sont pas reconnues : 404 sur <code>/api/...</code>
jersey-hk2	Jersey ne démarre pas (« <code>InjectionManagerFactory not found</code> »)
jersey-media-json-binding	erreur 500 « <code>MessageBodyWriter not found</code> » : Jersey ne sait pas écrire du JSON

6. Étape 3 : déclarer l'application REST

Créez le package `ma.youcode.workshop.resources`, puis la classe `RestApplication` :

```
1  package ma.youcode.workshop.resources;
2
3  import jakarta.ws.rs.ApplicationPath;
4  import jakarta.ws.rs.core.Application;
5
6  @ApplicationPath ("/api")
7  public class RestApplication extends Application {
8  }
9
```

7. Étape 4 : créer la ressource ApprenantsResource

Dans le package `ma.youcode.workshop.resources`, créez la classe `ApprenantsResource`.

Exercice 1 : lire tous les apprenants

Consigne : créez la classe `ApprenantsResource` avec une méthode qui répond à `GET /api/apprenants` et renvoie en JSON la liste de tous les apprenants.

Exercice 2 : lire un apprenant par son id

Consigne : ajoutez une méthode qui répond à `GET /api/apprenants/{id}` et renvoie l'apprenant correspondant.

Indications :

- Dans `ApprenantDao`, écrivez une méthode `Apprenant findById(int id)` :
 - requête `select * from etudiant where id = ?` avec un `PreparedStatement` ;
 - retournez `null` si aucune ligne n'est trouvée.
- Dans la ressource :
 - annotez la méthode avec `@GET` et `@Path("/{id}")` ;
 - récupérez l'id avec `@PathParam("id") int id` ;
 - retournez un objet `Response : Response.ok(apprenant).build()` si l'apprenant existe, sinon `Response.status(Response.Status.NOT_FOUND).build()`.

Résultats attendus :

Requête	Statut	Corps
<code>GET /api/apprenants/1</code>	200	<code>{"email":"sara@ecole.ma",...,"id":1,...}</code>
<code>GET /api/apprenants/999</code>	404	vide

Exercice 3 : créer un apprenant

Consigne : ajoutez une méthode qui répond à `POST /api/apprenants`, reçoit un apprenant en JSON et l'enregistre en base.

Résultats attendus :

- la réponse est **201 Created** ;
- un nouvel appel à `GET /api/apprenants` ;
- la page HTML `/apprenants` l'affiche aussi, car c'est la même base et le même DAO.

8. Étape 5 : construire et déployer

9. Étape 6 : tester l'API

9.1 Dans le navigateur

Ouvrez <http://localhost:8080/workshop-connection-db/api/apprenants>. Vous devez obtenir :

Réponse attendue (200 OK)

```
[
  {"email":"sara@ecole.ma","filiere":"Informatique","id":1,"nom":"Alaoui","prenom":"Sara"},
  {"email":"youssef@ecole.ma","filiere":"Gestion","id":2,"nom":"Bennani","prenom":"Youssef"}
]
```

Remarques : les propriétés sont triées par ordre alphabétique (comportement par défaut de JSON-B), et un champ `null` (par exemple un email vide) est simplement omis.

9.2 Avec curl

```
curl -i http://localhost:8080/workshop-connection-db/api/apprenants
```

L'option `-i` affiche les en-têtes : vérifiez `HTTP/1.1 200` et `Content-Type: application/json`.

9.3 Avec Postman

- Méthode : **GET**
- URL : <http://localhost:8080/workshop-connection-db/api/apprenants>